

ARROW ELECTRONICS · ARCHITECTURE BRIEFING

Private Databricks for Agents

Architecture alternatives · APIM as AI Gateway · Foundry and Copilot Studio in a private VNet

Thursday 1:00 - 4:00 PM, Michael Yaacoub · Friday 8:00 - 11:00 AM, David Barkol

Michael Yaacoub | Sr Solution Engineer

Two sessions: architecture on Thursday, security on Friday

THURSDAY · 1:00 - 4:00 PM · MICHAEL YAACOUB
Architecture, APIM operations and live demo

1:00 Objectives and Arrow context

1:10 Architecture alternatives for private Databricks

1:40 APIM as AI Gateway: reference architecture

2:05 Operationalizing APIM: global vs project scope

2:35 Break

2:45 DEMO: MCP, REST and Genie via APIM

3:25 DEMO: Foundry and Copilot Studio private VNet

3:50 Decision framework, owners and next steps

FRIDAY · 8:00 - 11:00 AM · DAVID BARKOL
MCP security deep dive

8:00 Recap and MCP threat model framing

8:30 APIM policies: authentication and authorization

9:15 Guardrails: enforcing business limits at the gateway

9:45 Break

10:00 Blocking direct access to the MCP server

10:40 Threat model review, owners and dates

Friday builds directly on Thursday's gateway

Thursday proves the pattern works end to end; Friday hardens it. Both days use the same gateway and the same demo environment.

Michael Yaacoub | Sr Solution Engineer

DAY ONE · THURSDAY 1:00 - 4:00 PM · MICHAEL YAACOUNB

Architecture, gateway operations, live demo

What we cover: the four architecture alternatives for reaching private Databricks data from Foundry and Copilot Studio; APIM as the AI Gateway and how to operationalise it across global and project scopes; and a working demo calling the Databricks MCP server, the Databricks REST API and the Genie API through APIM from both platforms.

Four ways to reach private Databricks data from an agent

Databricks stays private in every option. What changes is where governance lives and how far the pattern scales.

OPTION 1 · MANAGED ENVIRONMENT + VNET SUPPORT

Copilot Studio and Power Automate reach Databricks from a managed environment with VNet support enabled. No APIM.

Strength Simplest architecture, strong Power Platform fit
Watch Limited to supported connectors; governance is decentralised as integrations grow

OPTION 2 · API MANAGEMENT + DATABRICKS APIS AND MCP

Databricks stays private; APIM publishes governed REST and MCP endpoints that any agent can consume.

Strength Central gateway, reusable across many agents
Watch APIM infrastructure and API lifecycle to operate
Recommended general-purpose enterprise pattern

OPTION 3 · MICROSOFT FOUNDRY + PRIVATE VNET INJECTION

Foundry hosts the orchestration layer and injects into the VNet to reach Databricks and other private enterprise resources.

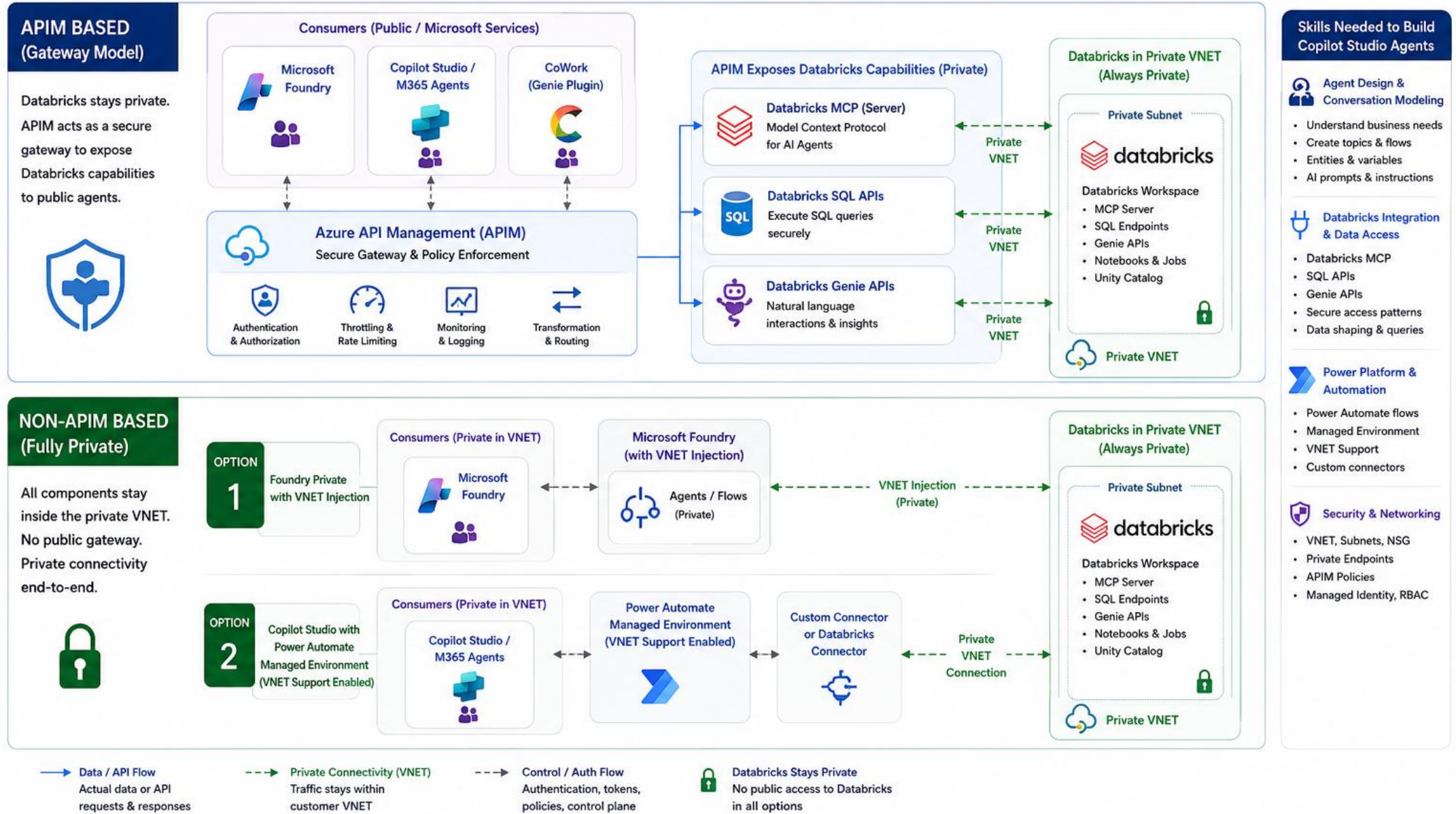
Strength Best for sophisticated agentic workflows
Watch Highest engineering complexity; more components to operate and observe

OPTION 4 · M365 COWORK + GENIE PLUGIN + APIM

M365 productivity surfaces call a Genie plugin, with APIM as the governed interface to Databricks data.

Strength Business-user friendly, reuses APIM governance
Watch Depends on plugin integration maturity; less flexible than Foundry for multi-agent orchestration

Options 2 and 4 share the same gateway. Choosing 2 today does not close the door on 3 or 4 later.





Microsoft

Pick by requirement, not by preference

Architecture

Best for

Key Benefits

When to use

OPTION

1

Managed Environment with VNet support

(Fully Private – No APIM)

Simple, single knowledge source scenarios or when you want the simplest, least new infrastructure

- ✓ Simplest architecture
- ✓ Least new infrastructure
- ✓ Fully private end-to-end
- ✓ Fastest to implement



Start here when needs are simple and governance requirements are light. Ideal for early projects or pilots.

OPTION

2

APIM with Databricks APIs and MCP

(APIM Based Gateway)

Enterprise API governance, multiple agents/apps and many future knowledge sources

- ✓ Centralised API governance
- ✓ Scalable for many consumers
- ✓ Reuse across many agents & apps
- ✓ Unified gateway for MCP, SQL, Genie



Choose when you need strong governance, control, observability and plan to add many knowledge sources.

OPTION

3

Foundry with Private VNet Injection

(Fully Private – No APIM)

Advanced agentic AI and complex orchestrations within Microsoft Foundry

- ✓ Highest performance & control
- ✓ Native to Foundry for complex flows
- ✓ Fully private end-to-end
- ✓ Ideal for advanced AI workloads



Use when workloads are complex, orchestrated in Foundry and require maximum capability and control.

OPTION

4

CoWork with Genie Plugin and APIM

(APIM Based Gateway)

M365 productivity scenarios where users interact via CoWork with Genie plugin

- ✓ Leverage CoWork + Genie experience
- ✓ Governed access via APIM
- ✓ Reuse Databricks capabilities
- ✓ Scalable and secure



Use when the primary experience is CoWork with Genie and you want governed, reusable access to Databricks data.



RECOMMENDATION

Start with Option 2 (APIM with Databricks APIs and MCP). It keeps Databricks private, puts governance in one place, and every additional knowledge source becomes another API or MCP tool rather than another integration. Then use Options 3 and 4 to build on the same gateway. Use Option 1 for simple, self-contained scenarios.



APIM
Gateway



Databricks
(Private VNet)



SUMMARY GUIDE

Simplicity & Speed

Option 1

Governance & Scale

Option 2

Advanced AI & Orchestration

Option 3

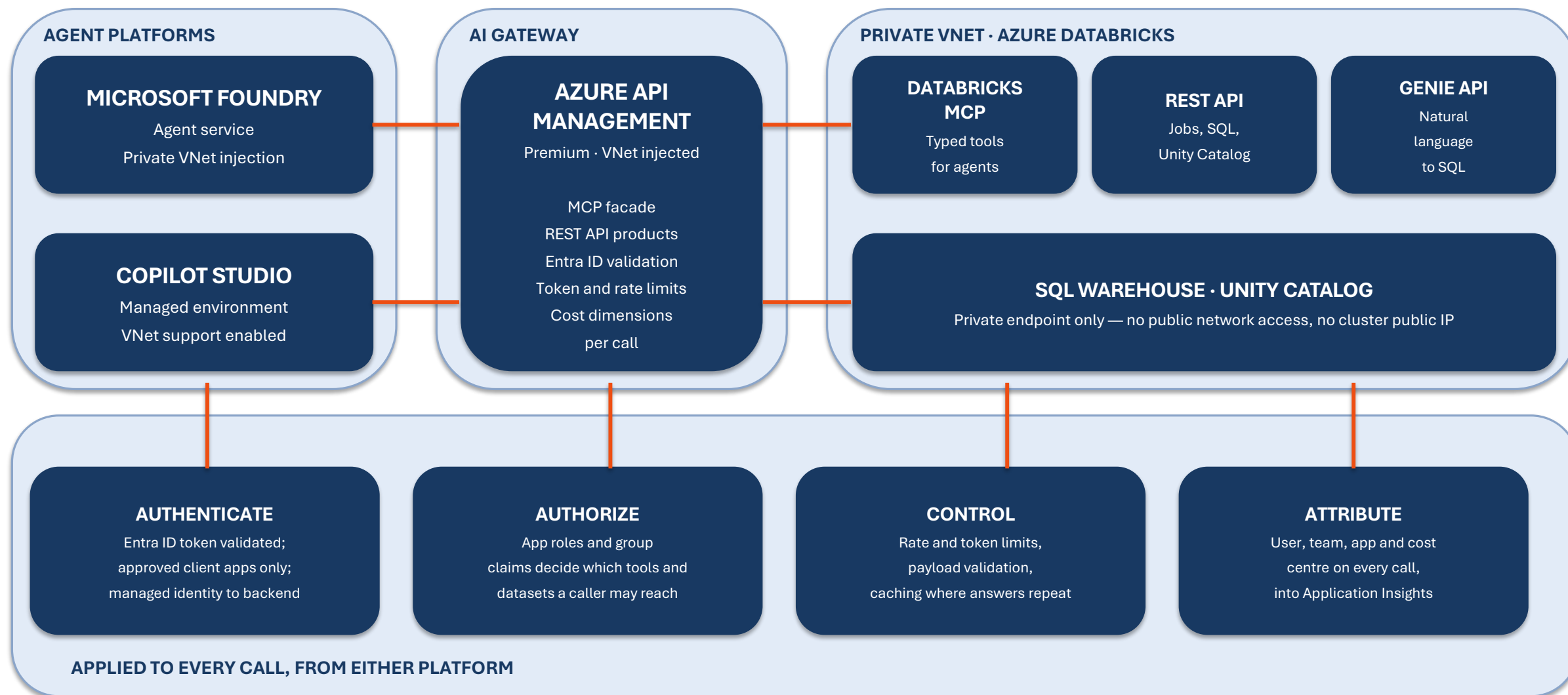
M365 Productivity

Option 4

APIM is the single front door to private Databricks

Two agent platforms, one governed gateway, three Databricks surfaces — all inside the private network.

PRIVATE NETWORK



Three Databricks surfaces, and when to reach for each

All three are published through the same APIM instance and inherit the same global policy.

DATABRICKS MCP

Tools an agent can discover and call by name, with typed arguments and schemas.

Use when the agent should take an action or run a known task and you want authorization at the level of the tool.

APIM role: MCP facade, app-role check per tool, argument validation before dispatch.

DATABRICKS REST API

Jobs, SQL statements, clusters and Unity Catalog metadata over plain HTTP.

Use when you need deterministic, parameterised access to a specific dataset or a specific job.

APIM role: import the OpenAPI, publish only the operations Arrow approves, then throttle.

GENIE API

Natural language to SQL over a curated Genie space, answering business questions directly.

Use when the user asks in their own words and the value is the answer itself, not the query behind it.

APIM role: the same gateway, plus semantic caching and a token budget per team.

THE POINT

One APIM instance, three backends, one identity model and one cost record. Foundry and Copilot Studio call whichever surface fits the job without either team learning a new network path or holding a Databricks credential.

Two ways to get the MCP server, same gateway either way

OPTION A · EXPOSE AN EXISTING API

No code. APIM exports an existing REST API as an MCP server and each selected operation becomes a tool. Fastest path when the invoicing API already exists.

Best when the API is stable and the tool surface maps one to one to operations.

OPTION B · BUILD FROM SCRATCH

A custom MCP server on Azure Functions or Container Apps, VNet integrated, then published through APIM as a governed passthrough MCP endpoint.

Best when tools need orchestration, validation or several backend systems.

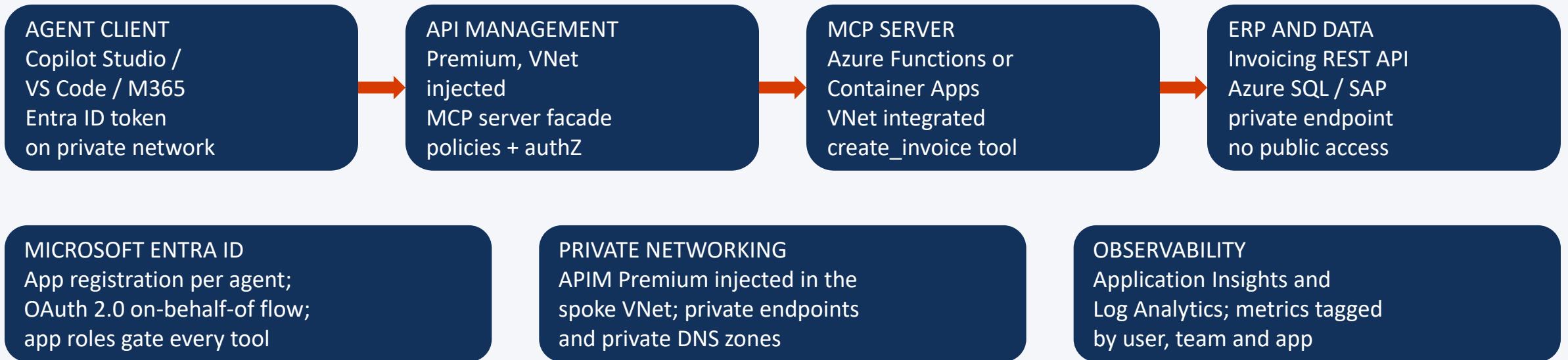
<https://github.com/Azure-Samples/remote-mcp-apim-functions-python>

<https://github.com/Azure-Samples/remote-mcp-apim-appservice-dotnet>

<https://github.com/Azure-Samples/AI-Gateway>

<https://learn.microsoft.com/azure/api-management/export-rest-mcp-server>

APIM fronts the invoicing MCP server inside the VNet

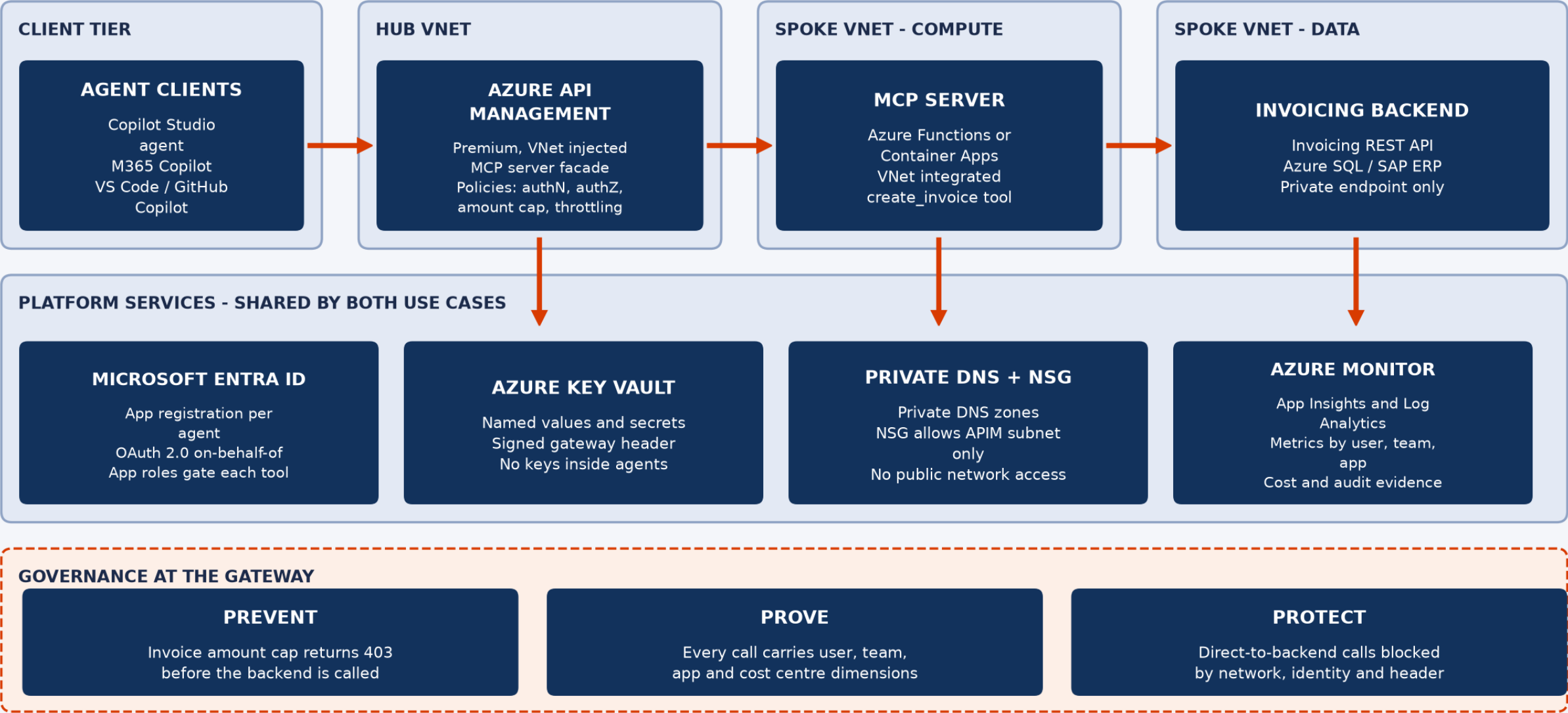


The agent never reaches the ERP directly - APIM is the only route in, and the only place guardrails live.

<https://learn.microsoft.com/azure/api-management/mcp-server-overview>

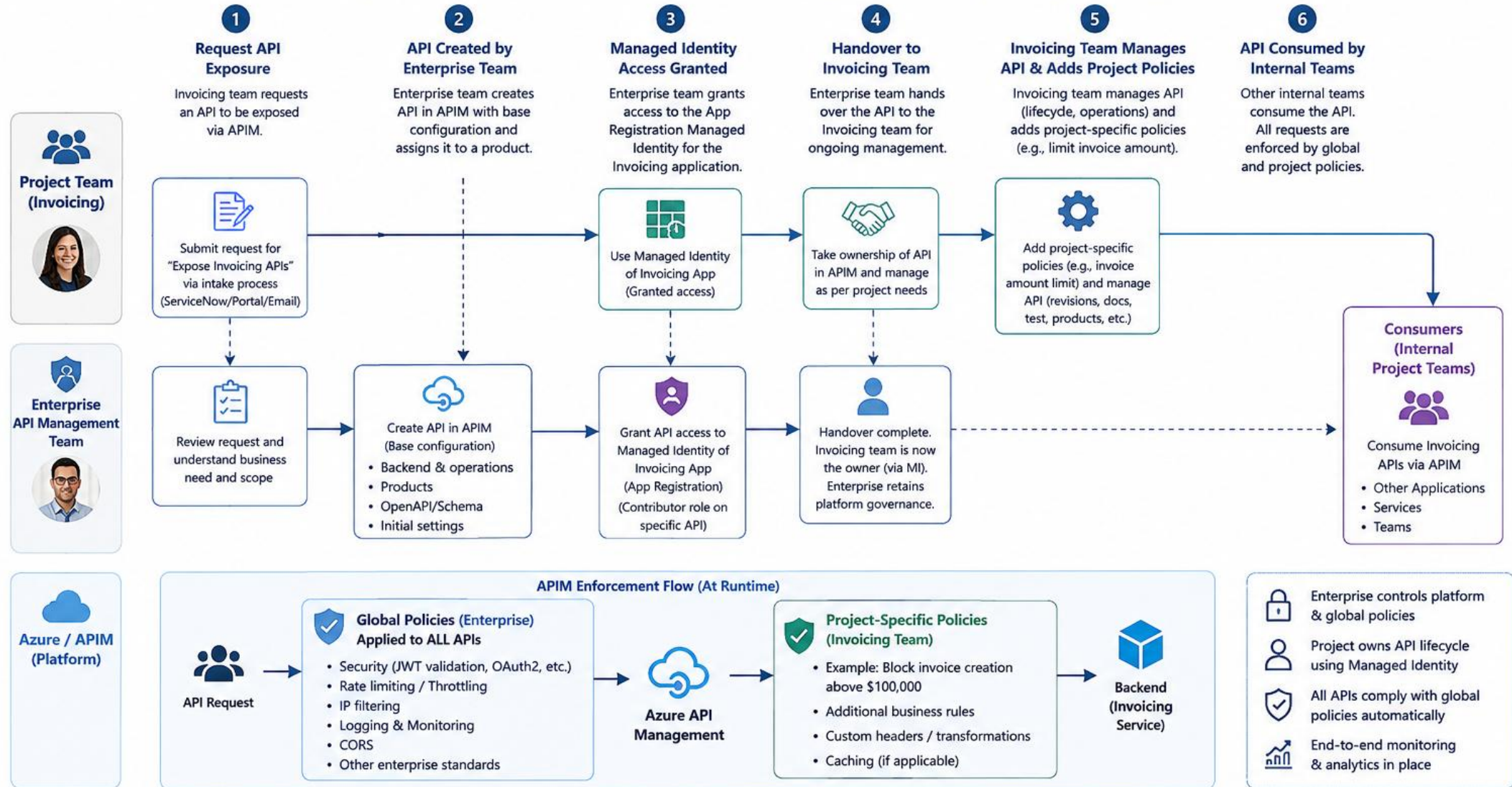
Use case 1 - Invoicing MCP server behind API Management

All traffic stays inside the corporate virtual network; the gateway is the only route to the invoicing backend.



Operational Process – Expose Invoicing APIs via APIM

Enterprise team maintains global APIM and policies. Project teams own and manage their APIs using a managed identity after access is granted.



✓ Outcome: Invoicing team can manage their API autonomously while ensuring compliance with enterprise standards and security.



Global policy is the platform team's contract; project policy is the team's freedom

APIM evaluates policy in four scopes. Deciding what lives in each scope is the operating model, not a configuration detail.

GLOBAL · ALL APIS · PLATFORM TEAM OWNS

- Entra ID token validation and tenant pinning
- Managed identity to every backend, no keys
- Baseline rate limit and payload size ceiling
- Correlation id and the signed gateway header
- Cost dimensions: user, team, app, cost centre
- Standard error shape and CORS posture
- Defender for APIs and diagnostic settings

Changed through the platform team's pipeline only.
No project can weaken these.

PRODUCT AND API · PROJECT TEAM OWNS

- Which tools and operations the product exposes
- App roles required for each operation
- Project-specific quota and token budget
- Backend routing, timeouts and retry policy
- Semantic caching where answers repeat
- Request and response shaping for the agent
- Project tags for chargeback

Changed by the owning team, reviewed against the global contract at merge.

ONE REPO, TWO FOLDERS

Global policies and project policies version together;
CI validates every change.

NAMED VALUES, NOT LITERALS

Thresholds, tenant ids and backend URLs come from Key Vault-backed named values.

BASE IS NOT OPTIONAL

Every project policy starts with `<base />` so the global contract always runs first.

The test: a new agent team should be able to ship without asking the platform team for a policy exception.

Private networking is configured once per platform, not per agent

MICROSOFT FOUNDRY · PRIVATE VNET INJECTION

Inject the Foundry project into a delegated subnet in the Arrow spoke VNet, so agent egress originates inside the network rather than from the public service.

What to configure

- Delegated subnet for agent compute
- Private endpoints for the AI services account and associated storage and search resources
- Private DNS zones so names resolve internally
- Outbound rules that allow the APIM subnet only
- Managed identity on the project, granted the gateway's app role - no keys in the agent

Result

The Foundry agent calls the APIM MCP and REST endpoints by private name, and Databricks never sees a public caller.

POWER PLATFORM · VIRTUAL NETWORK SUPPORT

Bind the managed environment to delegated subnets in paired regions, so Copilot Studio and Power Automate egress through Arrow's network.

What to configure

- Managed environment and an enterprise policy
- Two delegated subnets in paired regions
- Virtual network support enabled on the agent
- Entra ID single sign-on so the user's identity flows
- DLP policy scoping which connectors may be used
- On-premises data gateway on a VM as the fallback where injection is not available for a connector

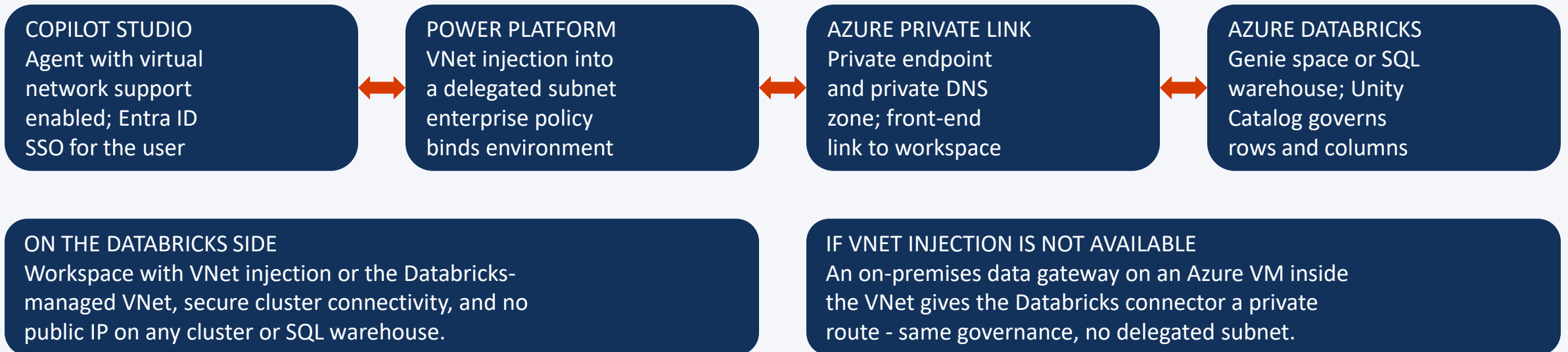
Result

The same governed APIM endpoints, reached from a business-user surface with the user's own permissions.

Both platforms land on the same gateway subnet, so one network review covers every agent Arrow builds after this.

<https://learn.microsoft.com/microsoft-copilot-studio/admin-network-isolation-vnet>

Copilot Studio reaches Databricks without a public IP

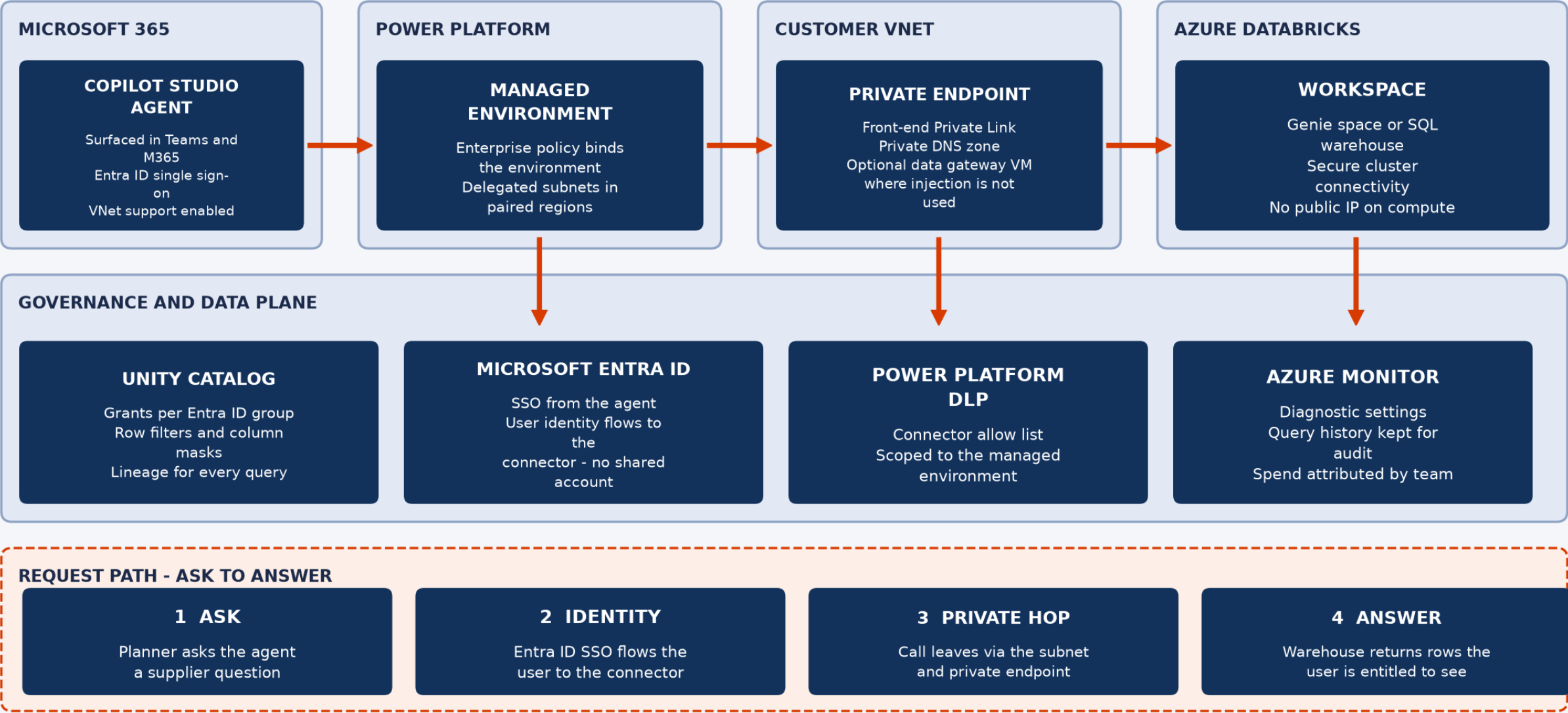


Unity Catalog stays the authority: the agent inherits the user's permissions rather than a shared service account.

<https://learn.microsoft.com/microsoft-copilot-studio/admin-network-isolation-vnet>

Use case 2 - Azure Databricks data in Copilot Studio over a private network

The agent reaches the lakehouse through a delegated subnet and a private endpoint; Databricks has no public endpoint.



Six things must be switched on before the demo works

POWER PLATFORM ENVIRONMENT

Managed environment, enterprise policy for VNet support, and two delegated subnets in paired regions.

NETWORKING

Delegated subnets, private endpoint to the workspace, private DNS zone, and firewall rules that allow only the required flows.

GOVERNANCE

Unity Catalog grants per group, row and column masks for supplier and price data, and lineage kept for every agent query.

COPILOT STUDIO AGENT

Virtual network support switched on, Entra ID authentication, and the Databricks connector added as a tool or knowledge source.

AZURE DATABRICKS

VNet-injected or managed workspace, front-end private link, secure cluster connectivity, and a serverless or classic SQL warehouse.

OPERATIONS

Diagnostic settings to Log Analytics, query history kept for audit, and DLP policies scoping which connectors the environment may use.

Networking and Databricks items are the long poles - start them in parallel with the agent build.

LIVE DEMO

Private Databricks, three ways, one gateway

Real data in Databricks. APIM in front of the Databricks MCP server, the Databricks REST API and the Genie API. We call all three from Microsoft Foundry and from Copilot Studio, over the private network, live.

<https://github.com/csdmichael/Azure-Databricks-Private-Agent-APIM>

Demo run of show: seven moves, forty minutes

1 THE DATA

Real tables in Databricks behind a private endpoint. Show that the workspace has no public access and the warehouse is reachable only from the VNet.

2 THE GATEWAY

Walk the APIM instance from the repo: the global policy fragment, then the three APIs - Databricks MCP, Databricks REST and Genie.

3 CALL MCP FROM FOUNDRY

Foundry agent discovers the tools through APIM, calls one, and returns a typed result - all from inside the injected subnet.

4 CALL THE REST API

A governed SQL statement against the warehouse, showing that only the operations Arrow approved are published on the product.

5 ASK GENIE FROM COPILOT STUDIO

A business question in natural language, answered from the lakehouse through the same gateway with the user's own permissions.

6 BREAK IT ON PURPOSE

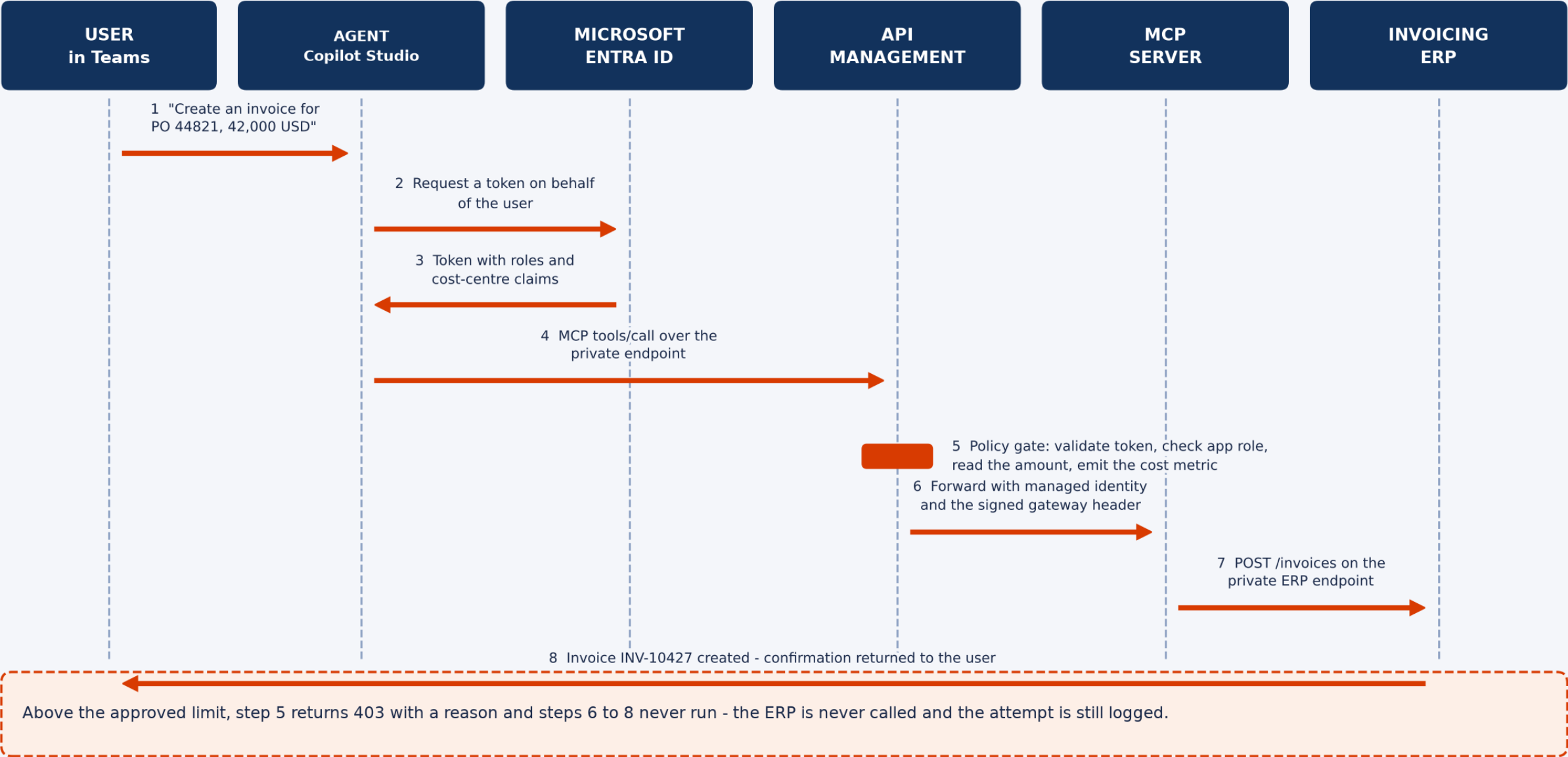
Call a backend directly, and with a token that lacks the role. Both fail at the gateway - the same failure David walks through on Friday.

7 SHOW THE RECEIPTS

Application Insights: every call from all three surfaces, attributed to a user, a team, an app and a cost centre.

End-to-end request - one invoice call, from the user to the ERP and back

The same eight steps run for every MCP tool call; only the policy decision at step 4 changes the outcome.



What the client actually sends, and what comes back

REQUEST · agent to the APIM MCP endpoint

```
POST https://apim-arrow.azure-api.net/  
mcp/databricks/messages  
Authorization: Bearer eyJ0eXAiOi...  
x-arrow-team: global-components
```

```
{  
  "jsonrpc": "2.0",  
  "id": 7,  
  "method": "tools/call",  
  "params": {  
    "name": "query_supplier_leadtime",  
    "arguments": {  
      "supplierId": "SUP-3390",  
      "quarter": "2026-Q2"  
    }  
  }  
}
```

RESPONSE · authorized caller

```
{  
  "jsonrpc": "2.0", "id": 7,  
  "result": { "content": [ {  
    "type": "text",  
    "text": "3 of 14 orders missed"  
  } ] }  
}
```

RESPONSE · caller lacks the app role

HTTP/1.1 403 Not authorized

```
{  
  "error": "Databricks.Query role  
required for this tool."  
}
```

Databricks is never reached on the second path.

Sample payloads for discussion - endpoint names and identifiers are illustrative.

A supplier question, answered without leaving the VNet

1 ASK

A planner asks the agent in Teams which suppliers missed their promised lead time last quarter.

2 IDENTITY

Copilot Studio authenticates with Entra ID SSO and flows that identity to the connector.

3 PRIVATE HOP

The call leaves through the delegated subnet over a private endpoint - never the public internet.

4 QUERY

A Databricks SQL warehouse runs the query; Unity Catalog trims data to the user's grants.

5 ANSWER

Results return over the same private path and the agent answers, citing the tables it used.

Same pattern for write-back: route the action through APIM as an MCP tool so the guardrails from use case 1 apply.

Every tool call carries who, which team and which app

```
<inbound>
<base />
<set-variable name="team" value="@context.Request
.Headers.GetValueOrDefault("x-arrow-team","none"))" />
<emit-metric name="McpToolCall" value="1"
namespace="arrow-agent-gateway">
<dimension name="User" value="@context.User?.Id)" />
<dimension name="Team"
value="@((string)context.Variables["team"])" />
<dimension name="App" value="@context.Product?.Name)" />
<dimension name="Tool" value="@context.Operation.Id)" />
</emit-metric>
</inbound>
```

BY USER

A dimension on the object id in the Entra ID token - answers who ran the tool, and when.

BY TEAM

A cost centre header or a group claim, always defaulted and never trusted blindly from the client.

BY APP

One APIM product per agent app: one product, one budget, one owner, one throttle.

For model traffic add llm-emit-token-metric with the same dimensions - tokens and spend land in the same view.

The app registration carries the FinOps metadata

```
{
  "displayName": "Arrow Invoicing Agent",
  "tags": [ "costCenter:CC-4821", "environment:prod",
    "businessUnit:global-components",
    "owner:integration-platform@arrow.com" ],
  "appRoles": [
    { "value": "Invoice.Create", "allowedMemberTypes": [ "Application", "User" ] }
  ]
}
```

TAG IT AT THE SOURCE

Cost centre + environment tags on the Azure resources, the app registration and the APIM product - one taxonomy.

SHOW IT BACK

Cost Management views and workbooks split gateway spend by cost centre; anomaly alerts catch runaway agents early.

MAKE IT STICK

A budget per APIM product with action groups, plus quarterly showback so every team sees what its agents cost.

DAY TWO · FRIDAY 8:00 - 11:00 AM · DAVID BARKOL, MCP SECURITY GBB

Securing the agent gateway

What we cover: the APIM policies that authenticate and authorize every MCP tool call; the guardrail that caps invoice value before the ERP is reached; and the six layers that stop a developer from calling the MCP server directly. Thursday's architecture is the input; this session hardens it.

Identity first: no keys, no anonymous callers, no shared accounts

Authentication, authorization and fair use, applied once at the gateway for every MCP tool and API.

```
<inbound>
<base />
<!-- Authentication: Entra ID tokens only, from approved agent apps -->
<validate-azure-ad-token tenant-id="{{entra-tenant-id}}" failed-validation-httpcode="401">
<client-application-ids>
<application-id>{{copilot-agent-app-id}}</application-id>
</client-application-ids>
<required-claims>
<claim name="roles" match="any"><value>Invoice.Create</value></claim>
</required-claims>
</validate-azure-ad-token>
<!-- Authorization to the backend: managed identity, never a shared key -->
<authentication-managed-identity resource="api://arrow-invoicing" />
<!-- Fair use: throttle per calling identity, not per subscription key -->
<rate-limit-by-key calls="60" renewal-period="60" counter-key="@context.User?.Id" />
</inbound>
```

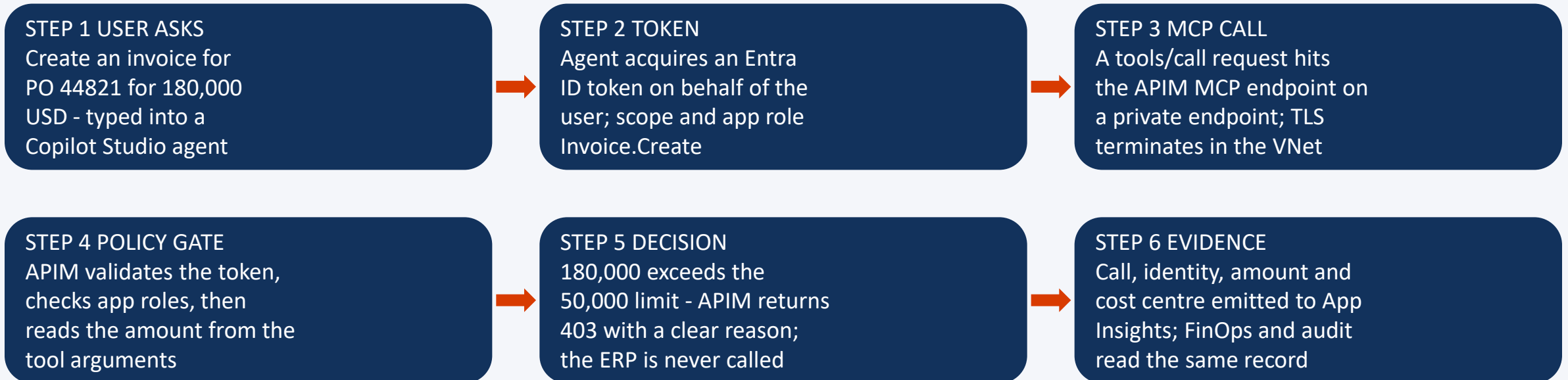


One inbound policy caps invoice value before the ERP is called

APIM inbound policy on the MCP tool operation - the threshold lives in the gateway, not in agent instructions.

```
<inbound>
<base />
<!-- Read the amount from the MCP tools/call arguments -->
<set-variable name="amount" value="@{
var body = context.Request.Body.As<JsonObject>(preserveContent: true);
return (decimal?)body["params"]?["arguments"]?["amount"] ?? 0m;
}" />
<!-- Hard stop above the approved threshold -->
<choose>
<when condition="@(((decimal)context.Variables["amount"]) > 50000m)">
<return-response>
<set-status code="403" reason="Invoice limit exceeded" />
<set-body>{"error":"Above the 50,000 USD limit. Route to manual approval."}</set-body>
</return-response>
</when>
</choose>
</inbound>
```

An over-limit invoice is stopped at the gateway, not the ERP



A request under the limit follows the identical path and reaches the ERP - one code path, one audit trail.

Amounts shown are illustrative thresholds for discussion, not Arrow policy limits.

A developer who bypasses the gateway must fail, not succeed quietly

The risk: a developer with the container URL calls `create_invoice` directly and every APIM guardrail is skipped.

LAYER 1 NETWORK

No public ingress at all. Private endpoint only, public network access disabled, and an NSG that allows the APIM subnet as the sole source.

LAYER 2 IDENTITY

The MCP app registration accepts only the gateway's managed identity. A developer token has no appid claim the backend will accept.

LAYER 3 PROOF OF ROUTE

APIM adds a signed header from Key Vault. The MCP server rejects any request without it, so a copied URL alone is not enough.

LAYER 4 PLATFORM POLICY

Azure Policy denies public IPs and public network access on the Functions, Container Apps and data resources in the workload subscription.

LAYER 5 DEVELOPER PATH

Developers get a sandbox APIM instance with the same policies and test data - the sanctioned route is the easiest route, so nobody needs a shortcut.

LAYER 6 DETECTION

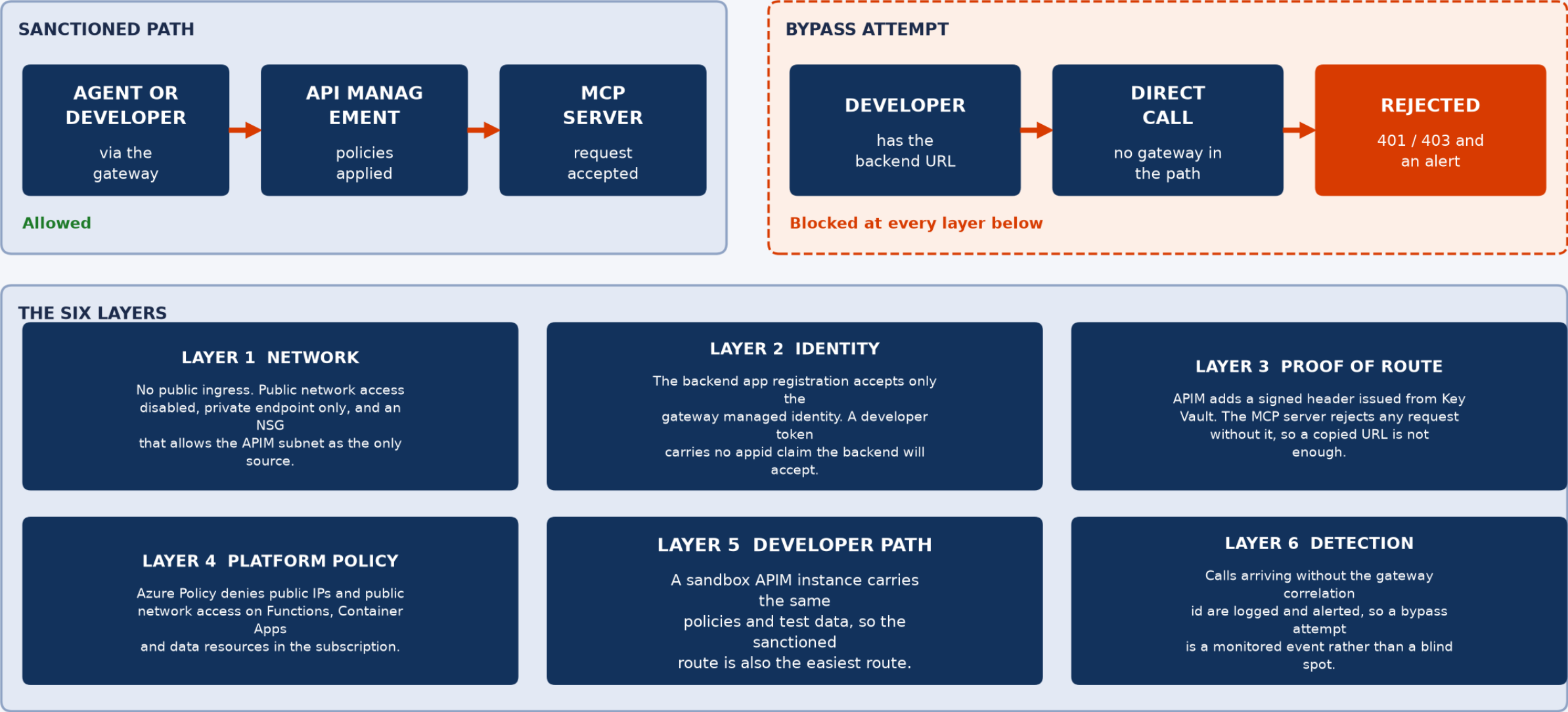
The MCP server logs any call without the gateway correlation id and alerts. Direct-access attempts become a monitored event, not a blind spot.

Network isolation alone is not enough - anyone inside the VNet could still reach the backend. Layers 2 and 3 close that gap.

<https://learn.microsoft.com/azure/api-management/secure-mcp-servers>

Blocking the bypass - a developer cannot reach the MCP server directly

Six independent layers; the direct path fails at the network, at identity and at the application.



Everything here is buildable from public samples and docs

GITHUB - START HERE

<https://github.com/Azure-Samples/AI-Gateway>

<https://github.com/Azure-Samples/remote-mcp-apim-functions-python>

<https://github.com/Azure-Samples/remote-mcp-apim-appservice-dotnet>

All three ship infrastructure as code

MICROSOFT LEARN

<https://learn.microsoft.com/azure/api-management/mcp-server-overview>

[.../api-management/secure-mcp-servers](#)
[.../api-management/export-rest-mcp-server](#)
[.../api-management/genai-gateway-capabilities](#)

<https://learn.microsoft.com/microsoft-copilot-studio/admin-network-isolation-vnet>

<https://learn.microsoft.com/azure/databricks/integrations/msft-power-platform/copilot-studio>

Links are current as of August 2026; the AI-Gateway repo is the fastest way to see the policies running.

Four decisions to leave Friday with

1 AGREE THE PILOT SCOPE

One invoicing tool and one supplier question, with owners and dates named before we leave the room.

2 STAND UP THE NETWORK

APIM Premium in the spoke VNet, delegated subnets and private endpoints - this is the long pole.

3 BUILD THE GUARDRAILS

Amount cap, app roles and cost dimensions kept in source control and deployed with the API.

4 PROVE THE SHOWBACK

One workbook that splits agent spend by user, team and app before access is widened.

Ask: name an owner for each of the four, and a date for the first private-network deployment.